

Dev Log — 25 July 2026

Session Goal

Work off the day's handwritten task list: clean up the input-system leftovers flagged at the end of the previous session, get the game actually running again (debug HUD rendering, snake moving), fix the crash-on-input regression that surfaced along the way, and — time permitting — start on `AppleSpawner`. As with recent sessions, the user wrote every code change by hand per `CLAUDE.md`'s read-only-for-the-agent policy; Claude's role was diagnosis, root-causing each crash/build failure with exact `file:line` citations, and describing fixes in prose. Landed in one commit (`b248412` , "orange build, game runs but doesn't work. Cleanup and fixing broken systems") — the message is accurate: the build compiles, links, and runs, but a mid-session architectural move (flattening `Snake`'s movement logic into `SnakeSegment` / `SnakeHead`) was left half-finished, so gameplay isn't actually correct yet. This log distinguishes what got fully closed out from what's still in progress, same as the previous session's log did for its own "wip" commit.

Work Completed

Input-system leftovers cleaned up (closes UE-0045/UE-0046/UE-0047)

- Deleted the dead duplicate `src/core/input/action-binding.cpp` — byte-identical leftover from the `ActionBindings` → `ActionRouter` rename, never in `CMakeLists.txt`, just a trap for anyone editing `action-router.cpp` without knowing its twin existed.
- `InputComponent::router` (`engine/input/components/input-component.hpp`) now has a `= nullptr` default member initializer, closing the indeterminate-pointer risk flagged last session.
- The misspelled, unused forward declaration `Core::Input::ActionRouters` (plural, never referenced) fixed to `ActionRouter`; `InputComponent`'s stray `final` dropped so it can be overridden if needed. The leftover commented-out `// , inputActions(snakeParams.inputActions)` line in `Snake`'s constructor initializer list — dead since `SnakeParams` dropped that member — was removed too.

Two `Core::Zip` crashes from a shared root cause, fixed one at a time

Added `ZeroValueFor(ActionValueType)` to `core/input/action-value.hpp` — a `switch` over `Boolean` / `Axis1D` / `Axis2D` returning a correctly-typed zero `ActionValue`. This exists because `ActionValue` is `std::variant<bool, float, Vector2D>`, and a bare default-constructed `ActionValue{}` always value-initializes to the *first* alternative (`bool`), regardless of what type the action actually is. Two call sites were doing exactly that and both hit `Core::Zip`'s `assert(a.index() == b.index())`:

- `KeyboardInputSource::GetActionValue`'s `std::accumulate` seed — crashed the moment any `Axis1D` / `Axis2D`-typed key (i.e. any movement key) was pressed, since the fold's `bool` seed got

Accumulated against a real `float` contribution. Fixed by seeding with `ZeroValueFor(action.type)` instead of `ActionValue{}`.

2. `ActionRouter::BindAxis`'s `AxisBinding::lastValue` initial value — crashed *instantly on frame one*, before any key press, because `ActionRouter::Update()`'s first `Distance(current, lastValue)` call compared a freshly-pollled `float / Vector2D` against the still-`bool` `lastValue`. Fixed the same way, seeding via `ZeroValueFor(this->actions.Get(name).type)`.

The user's own first attempt at `ZeroValueFor` didn't compile — it copy-pasted `ValueTypeOf`'s `std::visit` body without adapting the parameter, but `ZeroValueFor` takes an `ActionValueType` enum, not a real `std::variant`, and there's no `std::visit` overload for a plain enum. Confirmed by an actual compile attempt, not just inspection. Second pass switched on the enum directly, which compiles and is the version that shipped.

Debug HUD actually renders now

`Application` now resolves `Core::Debug::IDebugUserInterface` in `Initialize()` (previously declared as a member but never resolved — always null), and `OnDebugRender()` calls `debugUserInterface->RenderDebugHUD(Debug::System.GetActiveDebugHUD())`. Getting here took two iterations: the render call initially read a *different*, entirely dead member — `std::unique_ptr<Debug::DebugHUD> debugHUD` on `Application`, never constructed anywhere, a leftover from before the current `Debug::System`-based design — and dereferencing it null-segfaulted. Corrected to pull from `Debug::System`'s actual live HUD instance instead; the dead member was then deleted from `application.hpp` entirely. `Debug::DebugHUD` gained a `name` constructor parameter/field and `GetName()`, surfaced through `IDebugHUD / IDebugGameUI`, now logged on registration in `DebugUserInterfaceManager::Register`. `DebugConfig::showDebugLogs` was split into `showCoreDebugLogs / showClientDebugLogs` — only the former is actually wired to `LOG_INIT` so far.

Most keyboard input was silently dead — `MapKeyCode` filled in

`RaylibInputBackendFacade::MapKeyCode` only handled `A / D / S / W / Enter`; everything else (`E`, `Escape`, `Up`, `Down`, `Left`, `Right`) fell through to `return 0 (KEY_NULL)`, so arrow keys and the `Confirm / Decline` bindings were silently inert regardless of how correctly they were registered upstream. Found while diagnosing "most keys don't respond" — added the six missing cases.

Window double-free/segfault on clean exit, fixed across two false starts

`RaylibWindowFacade` now has a `closed` guard bool, checked in both the destructor and `CloseWindow()`. Root cause: `Application::Shutdown()` explicitly called `window->CloseWindow()`, and then `~RaylibWindowFacade()` called raylib's `::CloseWindow()` again during static teardown — raylib's `CloseWindow()` isn't idempotent, so the second call freed already-freed internal texture/shader/font state. Two attempts before this was actually fixed:

1. First pass guarded only the destructor — left the *public* `CloseWindow()` method (the one `Shutdown()` actually calls) still capable of firing unconditionally, so the double-free persisted, just with a different

heap-corruption signature (`double free or corruption (!prev)`) instead of `tcache 2`) because the crash now happened via a different call path.

2. Second pass added the guard to `CloseWindow()` too, but its body called the unqualified `CloseWindow()` — which resolved to itself (the member function), not raylib's `::CloseWindow()` — causing infinite recursion and a stack-overflow segfault instead. Diagnosed from the absence of raylib's own "TEXTURE unloaded / Window closed successfully" log lines, which confirmed raylib's real teardown was never reached at all.
3. Final version qualifies the real call as `::CloseWindow()` , with the guard set once, shared by both the destructor and the public method.

Snake/SnakeSegment/SnakeHead flattening — landed, but incomplete (see Known Issues)

Per the session's own notes ("Snake should be flattened into SnakeSegment and SnakeHead"), `Engine::Entities::Entity` gained a real `OnRegistration()` that adds a `TransformComponent2D` through the component system and caches a raw `transform` pointer — replacing the old pattern where `Entity` , `Apple` , and `SnakeSegment` each hand-rolled their own `make_unique<TransformComponent2D>(...)` independently. `IEntity::GetTransformComponent()` was removed from the interface entirely in favor of the cached pointer. `SnakeSegment` absorbed per-segment movement state that used to live centrally on `Snake` (`accumulatedDistance` , `speed` , `size` , `direction`) along with its own `Update()` / `Move()` / `SetDirection()` — each segment now advances itself instead of being pushed around by a central `Snake::Move()` .

`GameplayScene::Update()` now actually calls `this->snake->Update(deltaTime)` — previously never called from anywhere at all, which on its own was the reason the snake didn't move before any of this session's other fixes even applied. `GameplayScene::DebugUpdate()` / `DebugRender()` now forward into `entityManager` 's equivalents (previously empty no-ops).

This migration surfaced two linker errors — `SnakeHead::SetDirection` , `Snake::SetActive` both declared but "undefined reference" at link time. Root cause in both cases: the function bodies were commented out mid-edit, not missing or omitted from `CMakeLists.txt` — confirmed by reading the actual `.cpp` files rather than assuming a build-system problem. Restoring/rewriting the bodies against the new per-segment model is unfinished work — see Known Issues.

Duplicate frozen segment fixed

`Snake::CreateBody()` used to unconditionally call `CreateSegment()` even on the `i == 0` iteration, where a `SnakeHead` had already been constructed for that slot — producing `length + 1` body entries. `Snake::Move()` 's loop only ever covered indices `1..length-1` , so the extra tail entry rendered once at spawn and was never moved again: a static green square, easy to mistake for a frozen head rather than a duplicate segment. Fixed with a `continue` right after head construction so the `i == 0` iteration doesn't fall through into `CreateSegment()` too.

Off-topic but confirmed working: `ActionRouter::BindAxis` minor cleanup

While wiring in `ZeroValueFor`, `this->actions.Get(name)` — previously looked up twice in the same `BindAxis` call — is now captured into a local `action` and reused.

Lessons Learned

`std::variant` 's Default Constructor Always Picks the First Alternative

Both `Core::Zip` crashes this session trace back to the same C++ fact: a default-constructed `std::variant<bool, float, Vector2D>` is never "empty" or "unset" — it's always alternative `0` (`bool`, value `false`). Any code path that seeds/defaults an `ActionValue` with a bare `{}` is implicitly claiming "this is a boolean" whether or not that's true, and the mismatch only surfaces the moment it's compared or combined against a real value of a different alternative. Worth remembering as a variant-specific trap distinct from, say, `std::optional`, which does have a genuine empty state.

An Unqualified Call Inside a Member Function of the Same Name Resolves to Itself, Not the Global

`RaylibWindowFacade::CloseWindow()` 's infinite-recursion bug was caused by writing `CloseWindow();` inside the member function body — unqualified name lookup finds the enclosing class's own method before it ever considers the global `::CloseWindow()`, even though the intent was obviously to call raylib's function. This is a general C++ shadowing trap whenever a facade/wrapper method shares its name with the underlying library call it's wrapping — the `::` qualifier isn't optional style, it's load-bearing the moment names collide like this.

Diagnosing "No Log Output" Is as Informative as Diagnosing a Stack Trace

The infinite-recursion segfault produced no application-level crash message and, critically, none of raylib's own teardown log lines ("TEXTURE unloaded", "Window closed successfully") — which was itself the diagnostic signal that raylib's real `::CloseWindow()` was never reached at all, pointing straight at the self-referential call rather than needing a debugger/backtrace to find it.

Movement-Tick / Input-Sampling Coupling, and Why It Matters for Perceived Responsiveness

A design discussion mid-session (prompted by "A/D feel extremely unresponsive") traced actual input latency through the whole pipeline — `ActionRouter` 's threshold/ `Distance` change-detection (default `0.01f`, cleared instantly by any real key-down) and 60fps polling were both ruled out as non-issues by direct inspection of `Distance()` 's implementation and `WindowConfig::targetFPS`. The real cause was `Snake::Update()` calling `SetDirection()` every frame rather than once per movement tick (`speed = size * 5.0f` $\Rightarrow$ a 200ms tick interval) — direction changes registered instantly in the input system but were only visible once per tick, and a same-tick reversal could get silently overwritten by a later frame's read

before the tick ever consumed it, producing both sluggish feel and a diagonal-direction glitch when switching axes quickly (the two axis callbacks each only wrote their own component, so a stale opposite-axis value could linger uncleared). Agreed direction: latch only the single most-recent commanded direction (a `pendingDirection`-shaped slot, overwrite — not append — on each keypress), and consume-and-clear it once per tick rather than reading live state every frame. This is a single latched slot, not a true FIFO queue, since only the most recent command since the last tick consumption matters.

Y-Axis Convention: Isolate the Flip at the Rendering Boundary, Not in Gameplay Logic

A separate discussion (wanting `+Y` to mean "up" throughout the engine, more intuitive than raylib's screen-space `+Y = down`) confirmed there's currently no camera/viewport transform layer anywhere in the codebase — rendering is 1:1 pixel-for-unit straight into raylib calls. Established that the correct seam for the flip is the raylib-facade rendering boundary itself (`RaylibRendererFacade::DrawRectangle` , and equivalently the UI/debug-HUD facades if all screen elements should honor the convention), not `Snake::Move() / SetDirection()` or the key bindings — which turned out to already be written in the target math convention (`W / Up: +1.0f`) and only *look* inverted today because nothing flips at render time yet. No reverse (screen → engine) conversion is needed, since this game has no mouse input anywhere.

Architectural Decisions

`Entity::OnRegistration()` Centralizes `TransformComponent2D` Construction

Previously every `Entity` subclass (`Entity` itself, `Apple` , `SnakeSegment`) independently hand-rolled a `make_unique<TransformComponent2D>(...)` call in its own constructor or `OnRegistration` . Moved this into a single `Entity::OnRegistration()` that adds the component through the existing component-registration system and caches a raw `transform` pointer for subclasses to use — subclasses now call `Entity::OnRegistration()` first and build on top, rather than each re-deriving the same construction logic. `IEntity::GetTransformComponent()` was removed from the interface entirely in favor of this cached member, since nothing outside the entity itself needs polymorphic access to it once it's a plain cached pointer.

Movement State and Logic Move From Central `Snake` to Per-`SnakeSegment`

Per the session's own plan ("Snake should be flattened into `SnakeSegment` and `SnakeHead`"), `accumulatedDistance / speed / size / direction` and the `Update() / Move() / SetDirection()` methods that used to live solely on `Snake` , orchestrating every segment's position from one place, moved down onto `SnakeSegment` itself — each segment now owns and advances its own movement state. This is a deliberate decentralization, not yet fully realized in the shipped commit (`Snake` 's own versions of these methods are commented out mid-migration, not deleted) — see Known Issues for the concrete finishing work.

`pendingDirection` : a Single Latched Slot, Not a FIFO Queue

Decided (see Lessons Learned) that the fix for movement-tick/input coupling should latch only the single most recently commanded direction — last-write-wins — rather than a true multi-entry queue that processes every intermediate command in order. Since only the freshest input at tick-consumption time matters for this game, a `Vector2D` slot that gets overwritten on each axis-change callback is sufficient and avoids unnecessary queue/liveness bookkeeping. `SnakeHead::pendingDirection` exists in the shipped code as the starting point for this, declared but not yet read or written anywhere.

Known Issues Carried Forward

- **UE-0004** — BUG: Window-side event bus stub. Unchanged — `WindowResizedEvent` still an empty struct with no publisher.
- **UE-0014** — REAPER: `GameplayState` still owns stale snake/apple references and polls input. Unchanged across several sessions now — `GameplayState::snake / apple` are still never assigned; the dead `IsActionPressed` block remains commented out on a path that never runs.
- **UE-0015** — REAPER: `UserInterfaceManager` update/debug methods commented out. Unchanged.
- **UE-0016** — REAPER: `SnakeGameSettings::GetBoxSize()` commented out. Unchanged.
- **UE-0019** — SIDE QUEST: `BoundaryWrapSystem`. Unchanged — boundary wrapping is still handled inline (now inside `Snake::Teleport`, itself currently commented out mid-refactor, see UE-0048).
- **UE-0020/UE-0023** — REAPER: Remove stale `stateMachine` `shared_ptr` from `Application` members. Not investigated this session.
- **UE-0021** — 1UP: `std::is_abstract_v` discriminator breaks for `ColliderComponent2D`. Not investigated this session.
- **UE-0024** — REAPER: UI component system commented out. Not investigated this session.
- **UE-0026** — REAPER: Remove `Apple` verification hooks before merge. Still present — `Apple::OnActivate()` still does the `SetActive(false)` → `SetActive(true)` toggle-and-log verification dance, untouched this session.
- **UE-0028** — MAIN QUEST: `GameplayStateMachine` use-after-move. Unchanged, still entangled with UE-0035 below.
- **UE-0029** — RAID BOSS: Unify entity/component ownership to remove the leak and double free. Unchanged in scope — `RenderComponentUIManager::Register / Unregister` (separate from `RenderComponent2DManager`) still not addressed. This session's `Entity::OnRegistration()` work touched transform-component ownership, not this ticket's remaining scope.

- **UE-0030** — 1UP: `TransformComponent2D::GetTransform()` returns a shared function-local `static`. Not touched this session.
 - **UE-0031** — REAPER: Dependency injector correctness bugs. Not touched this session.
 - **UE-0032** — REAPER: `ComponentDispatcher` `const_cast` ing. Not touched this session.
 - **UE-0033** — REAPER: Miscellaneous cleanups from code review. Not touched directly this session; note sub-item (c), `Snake::Move` indexing `body` with `this->length`, is likely moot now that `Snake::Move()`'s body is commented out pending the per-segment refactor (UE-0048) — re-verify once that lands rather than fixing the old code as written.
 - **UE-0036** — Event's `action` field never populated. Flagged for confirmation two sessions running (07-23, and unconfirmed since) — the diff evidence for this being fixed is from the 07-23 session and wasn't touched again this session. Recommend confirming and closing next session rather than carrying it a third time on faith.
 - **UE-0037** — Keymap array literal order didn't match `Action` enum order. Recommended closing as superseded by the `ActionSet` /named-registration redesign since 07-23; not re-confirmed since. Same recommendation as UE-0036 — resolve the flag next session.
 - **UE-0051** — RAID BOSS: Debug System needs a full refactor Called out explicitly in this session's own handwritten planning notes ("the entire Debug System needs a big refactor") after getting the HUD rendering again via targeted, incremental wiring (resolving `debugUserInterface`, adding `DebugHUD::GetName()`) rather than a structural change. `Debug::System`'s relationship to `Application`'s now-mostly-vestigial `debugHud` member, the `DebugConfig::showCoreDebugLogs` / `showClientDebugLogs` split (only the former actually wired to anything), and the overall `IDebugHUD` / `IDebugUserInterface` / `IDebugGameUI` split are all candidates for this — needs its own dedicated session to scope properly rather than being chipped at incrementally.
-

Next Session

- **UE-0048** — MAIN QUEST: Finish the Snake → SnakeSegment/SnakeHead movement migration This session's flattening move landed the shape (per-segment `accumulatedDistance` / `speed` / `direction` / `Update` / `Move` / `SetDirection`) but left `Snake`'s own `Update()`, `Move()`, `CheckIfShouldGrow()`, `Teleport()`, `GetCenter()`, and `SetActive()` bodies commented out mid-refactor rather than removed or rewritten — two of them (`GetCenter`, `SetActive`) had to have their commented-out bodies restored just to fix a linker error this session, without yet being reconciled against the new per-segment ownership model. `SnakeHead::SetDirection` similarly just forwards to `SnakeSegment::SetDirection` with the real reversal-guard logic commented out beneath it, unused. Needs a real pass deciding what stays on `Snake` (orchestration across segments — growth, teleport-wrapping, active-toggling) versus what's now purely per-segment, and getting every method's body to match the new model rather than sitting half commented-out.

- **UE-0049** — MAIN QUEST: Implement the latched `pendingDirection` command queue Design agreed this session (see Lessons Learned / Architectural Decisions): `SnakeHead`'s two axis-bind callbacks should write into a single `pendingDirection` slot (already declared, currently unused) instead of `SetDirection` being invoked every frame. `Snake` /segment `Update()` should consume-and-clear that slot only inside the tick-boundary block (once per ~200ms movement tick), not every frame. This fixes both the perceived A/D sluggishness and the diagonal-direction glitch from a stale, un-zeroed opposite axis lingering across quick direction switches. Depends on UE-0048 landing first, since the tick-boundary block currently lives in code that's mid-rewrite.
- **UE-0050** — SIDE QUEST: Wire `showClientDebugLogs` (and generalize `showCoreDebugLogs`) into the logger properly `DebugConfig::showDebugLogs` was split into `showCoreDebugLogs` / `showClientDebugLogs` this session, but only `showCoreDebugLogs` is actually read anywhere (`LOG_INIT` in `Application::Run()`). A `// SIDE QUEST` comment left in the diff itself flags the intended follow-up: pass the whole `debug` settings struct into log setup so each logger's debug verbosity can be configured individually, with `debug.enabled` acting as a master switch rather than the current single flag.
- **UE-0038** — MAIN QUEST: Migrate `Confirm` / `Decline` off the dead `InputSystem::IsActionPressed` API. Unchanged this session — `main-menu-state.cpp:31` / `game-over-state.cpp:32` still weren't touched. `ActionRouter::Bind` is proven working (this is exactly what `SnakeHead` uses for axes, via `BindAxis`) — whatever owns `MainMenuState` / `GameOverState` needs its own `InputComponent` (or equivalent `Bind` call) the same way `SnakeHead` now does.
- **UE-0035** — MAIN QUEST: `GameContext::input` is never assigned; reconcile the `MainMenuState` workaround. Unchanged, still blocked on UE-0038 landing (plus UE-0014, UE-0028).
- **UE-0009** — MAIN QUEST: Implement `AppleSpawner`. Untouched this session — `Apple` is still hardcoded to `Vector2D::Zero()` in `OnRegistration()`, with `SetPosition(GetNewPosition())` commented out. Last item on this session's own handwritten task list, not reached.